



UNIVERSIDADE FEDERAL DO CEARÁ
CAMPUS DE QUIXADÁ
CURSO DE GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

VITOR MANOEL ALVES DA SILVA

**PROJETO E AVALIAÇÃO EXPERIMENTAL DE UMA CAMADA ARQUITETURAL
DESACOPLADA AGÊNTICA (*HARNESS*) PARA INTEGRAÇÃO DE MODELOS DE IA
EM APLICAÇÕES WEB: CUSTO DE SUBSTITUIÇÃO DE PROVEDOR E CUSTO
ENERGÉTICO**

QUIXADÁ

2026

VITOR MANOEL ALVES DA SILVA

PROJETO E AVALIAÇÃO EXPERIMENTAL DE UMA CAMADA ARQUITETURAL
DESACOPLADA AGÊNTICA (*HARNESS*) PARA INTEGRAÇÃO DE MODELOS DE IA EM
APLICAÇÕES WEB: CUSTO DE SUBSTITUIÇÃO DE PROVEDOR E CUSTO
ENERGÉTICO

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Engenharia de Software
do Campus de Quixadá da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Engenharia de Software.

Orientador: Prof. Dr. Sidarta Carvalho

QUIXADÁ

2026

VITOR MANOEL ALVES DA SILVA

PROJETO E AVALIAÇÃO EXPERIMENTAL DE UMA CAMADA ARQUITETURAL
DESACOPLADA AGÊNTICA (*HARNESS*) PARA INTEGRAÇÃO DE MODELOS DE IA EM
APLICAÇÕES WEB: CUSTO DE SUBSTITUIÇÃO DE PROVEDOR E CUSTO
ENERGÉTICO

Trabalho de Conclusão de Curso apresentado ao
Curso de Graduação em Engenharia de Software
do Campus de Quixadá da Universidade Federal
do Ceará, como requisito parcial à obtenção do
grau de bacharel em Engenharia de Software.

Aprovada em:

BANCA EXAMINADORA

Prof. Dr. Sidarta Carvalho (Orientador)
Universidade Federal do Ceará (UFC)

RESUMO

A integração de modelos de linguagem de grande escala a aplicações *web* costuma ser feita por chamadas diretas ao kit de desenvolvimento do provedor, espalhadas pelo código de negócio, o que acopla a aplicação a um fornecedor específico. Uma alternativa é interpor uma camada arquitetural intermediária — aqui denominada *harness* — que centraliza a comunicação com os provedores, a execução de ferramentas, o estado, o tratamento de falhas e o registro de eventos. A literatura propõe arquiteturas desse tipo e discute seus compromissos de forma qualitativa, mas não os mede em estudo controlado. Este trabalho avalia experimentalmente o *trade-off* entre desacoplamento e eficiência energética introduzido por essa camada, comparando duas versões funcionalmente equivalentes de uma mesma aplicação *web*: uma integrada ao provedor por meio do *harness* e outra integrada diretamente. O custo de substituição do provedor é quantificado por métricas comportamentais e estruturais do código, e o custo energético, por telemetria de *hardware* em requisições simples e agênticas a um modelo aberto executado localmente. Espera-se caracterizar em que condições o desacoplamento obtido compensa o custo energético da camada, oferecendo evidência quantitativa para uma decisão arquitetural hoje tomada com base em princípios de projeto ou em alegações de fornecedores.

Palavras-chave: Arquitetura de software. Integração de LLM. Substituibilidade. Eficiência energética. Green AI.

ABSTRACT

Large language models are usually integrated into web applications through direct calls to the provider’s software development kit, scattered across the business code, which couples the application to a specific vendor. An alternative is to place an intermediate architectural layer — here called a *harness* — that centralizes communication with providers, tool execution, state, failure handling, and event logging. The literature proposes such architectures and discusses their trade-offs qualitatively, but does not measure them in controlled studies. This work experimentally evaluates the trade-off between decoupling and energy efficiency introduced by this layer, comparing two functionally equivalent versions of the same web application: one integrated with the provider through the harness and one integrated directly. Provider replacement cost is quantified through behavioral and structural code metrics, and energy cost through hardware telemetry for simple and agentic requests to an open model running locally. The work is expected to characterize under which conditions the decoupling achieved outweighs the energy cost of the layer, providing quantitative evidence for an architectural decision currently made on the basis of design principles or vendor claims.

Keywords: Software architecture. LLM integration. Substitutability. Energy efficiency. Green AI.

LISTA DE ABREVIATURAS E SIGLAS

API *Application Programming Interface*

GPU *Graphics Processing Unit*

IA *Inteligência Artificial*

LLM *Large Language Model*

SDK *Software Development Kit*

SUMÁRIO

1	INTRODUÇÃO	7
1.1	Contexto e motivação	7
1.2	Problema e lacuna de pesquisa	9
1.3	Justificativa	10
1.4	Objetivos	11
1.4.1	<i>Objetivo geral</i>	11
1.4.2	<i>Objetivos específicos</i>	11
	REFERÊNCIAS	12

1 INTRODUÇÃO

1.1 Contexto e motivação

Os modelos de linguagem de grande escala (*Large Language Models*, LLMs) deixaram de ser objeto restrito de pesquisa e passaram a compor o cotidiano do desenvolvimento de *software*. Na pesquisa anual da plataforma Stack Overflow, a proporção de desenvolvedores que usam ou pretendem usar ferramentas de inteligência artificial (IA) subiu de 76% em 2024 para 84% em 2025, e 31% dos respondentes já declaravam usar agentes de IA (STACK OVERFLOW, 2025). O movimento não se limita às ferramentas de apoio ao programador: os próprios produtos passaram a incorporar modelos. Segundo levantamento da Menlo Ventures com líderes técnicos de empresas e *startups*, o gasto corporativo com interfaces de programação de aplicações (*Application Programming Interfaces*, APIs) de LLMs mais que dobrou em seis meses, de 3,5 bilhões de dólares em novembro de 2024 para 8,4 bilhões em meados de 2025 (MENLO VENTURES, 2025).

Em uma aplicação *web*, essa incorporação costuma ocorrer da forma mais direta possível: o código de negócio importa o kit de desenvolvimento (*Software Development Kit*, SDK) do provedor escolhido e chama o modelo nos pontos em que precisa dele. Cada provedor, porém, define o próprio formato de mensagens, de declaração e de chamada de ferramentas, de erros e de limites de uso. À medida que as integrações deixam de ser uma pergunta isolada ao modelo e passam a envolver a execução de ferramentas externas em ciclos de várias etapas — o que Sapkota *et al.* (2026) chamam de agentes de IA, em contraste com sistemas multiagentes de IA *agêntica* —, a quantidade de código dependente dessas convenções cresce. Esse tipo de dependência já foi descrito como dívida técnica em sistemas de aprendizado de máquina: o *glue code* que adapta a aplicação à API de um pacote específico tende a prender o sistema às particularidades desse pacote e a encarecer qualquer substituição posterior (SCULLEY *et al.*, 2015).

A Engenharia de Software conhece há décadas a resposta de princípio para esse problema. Parnas (1972) propôs que as decisões de projeto com maior probabilidade de mudar fossem encapsuladas em módulos que as escondessem do restante do sistema, e a modificabilidade é hoje tratada como um atributo de qualidade arquitetural a ser deliberadamente projetado (BASS *et al.*, 2021). Fora do domínio da IA, a dependência de um único fornecedor (*vendor lock-in*) é um problema arquitetural consolidado: a literatura de computação em nuvem reúne

um amplo conjunto de abordagens de interoperabilidade e portabilidade baseadas em camadas intermediárias (KAUR *et al.*, 2017), a noção de *middleware* como camada reutilizável que absorve a heterogeneidade entre plataformas está bem estabelecida (ISSARNY *et al.*, 2007), e há comparações empíricas entre funções *serverless* escritas sobre bibliotecas de abstração e sobre as APIs nativas do provedor (MO *et al.*, 2023).

No caso dos modelos de IA, a escolha do provedor é justamente uma decisão com alta probabilidade de mudar. O mesmo levantamento da Menlo Ventures registra que a participação da OpenAI no uso corporativo de LLMs caiu de 50%, ao fim de 2023, para 25% em meados de 2025, enquanto a Anthropic chegou a 32% e o Google a 20% (MENLO VENTURES, 2025). Ainda assim, apenas 11% das equipes consultadas trocaram de fornecedor no último ano, ao passo que 66% apenas atualizaram o modelo dentro do provedor que já utilizavam (MENLO VENTURES, 2025). Esses números não isolam a causa da baixa taxa de troca — desempenho, contratos e preço também pesam —, mas são coerentes com a hipótese de que trocar de provedor tem custo técnico relevante para quem integrou o modelo diretamente ao código.

Uma alternativa arquitetural é interpor, entre a aplicação e o provedor, uma camada dedicada à integração com os modelos. Na prática recente, o termo *harness* passou a designar a camada de *software* que envolve o modelo e se responsabiliza pela execução de ferramentas, pelo gerenciamento de estado e pela recuperação de erros. Neste trabalho, *harness* é definido operacionalmente como a **camada arquitetural intermediária entre a aplicação web e o provedor de IA**, que abstrai e centraliza as interações da aplicação com modelos e agentes, de modo que a aplicação não acessa diretamente os SDKs dos provedores. Atribuem-se a essa camada a padronização da comunicação com os provedores, a orquestração de chamadas e a execução controlada de ferramentas, o gerenciamento de contexto e estado, o tratamento de falhas e o registro estruturado de eventos; ela não implementa regras de negócio nem treina modelos. Essas responsabilidades dialogam com o catálogo de padrões arquiteturais para agentes baseados em modelos de fundação proposto por Liu *et al.* (2025), que serve de referência para o projeto da camada.

Essa camada, contudo, não é gratuita. Cada nível de indireção acrescenta trabalho a toda requisição — normalização de mensagens, serialização, orquestração do ciclo de ferramentas, registro de eventos —, e esse trabalho consome energia. A questão ganhou peso com a escala atual da IA. A Agência Internacional de Energia estima que os centros de dados consumiram cerca de 415 TWh em 2024, aproximadamente 1,5% da eletricidade mundial, e projeta que esse

consumo mais que dobre até 2030, chegando a cerca de 945 TWh, com a IA como principal vetor de crescimento (INTERNATIONAL ENERGY AGENCY, 2025). A área de *Green AI* propõe tratar a eficiência energética como critério de avaliação ao lado da acurácia (SCHWARTZ *et al.*, 2020), depois que Strubell *et al.* (2019) chamaram atenção para o custo energético e ambiental do treinamento de modelos de linguagem.

Na fase de uso, que se repete a cada requisição, os números também são expressivos. Luccioni *et al.* (2024) mostram que arquiteturas generativas de propósito geral chegam a ser ordens de grandeza mais custosas que sistemas específicos para a mesma tarefa: gerar texto consumiu em média 0,047 kWh a cada mil inferências, contra 0,002 kWh da classificação de texto. Além disso, o consumo depende fortemente da pilha de *software*, e não apenas do modelo: Fernandez *et al.* (2025) verificaram que estimativas baseadas em contagem de operações de ponto flutuante subestimam o consumo real e que otimizações bem aplicadas reduzem a energia total em até 73%. Vellaisamy *et al.* (2026) decompõem a sobrecarga de orquestração executada no processador hospedeiro, fora da unidade de processamento gráfico (GPU), e mostram que ela é um custo oculto da inferência, enquanto Ifath e Haque (2026) caracterizam os compromissos entre desempenho e energia em fluxos agênticos de múltiplas requisições, o mesmo padrão de uso que uma camada de *harness* orquestra. Decorre daí que uma decisão puramente arquitetural, como acrescentar uma camada entre a aplicação e o modelo, pode ter consequências energéticas mensuráveis, que raramente entram na avaliação dessa decisão.

1.2 Problema e lacuna de pesquisa

A literatura já propõe arquiteturas para integrar modelos e agentes a sistemas de *software* e discute os compromissos envolvidos. O catálogo de Liu *et al.* (2025) descreve, para cada padrão, contexto, forças e *trade-offs*, mas o faz qualitativamente, a partir de revisão de literatura. Kandogan *et al.* (2025) propõem uma arquitetura de referência para IA composta, com registros de agentes e de dados que abstraem provedores e APIs proprietárias, ilustrada por um caso de uso, sem comparação controlada com uma integração direta. Zhu *et al.* (2026), ao comparar *frameworks* de orquestração de agentes baseados em LLMs, destacam que parte relevante das alegações sobre esses *frameworks* provém dos próprios fornecedores, e não de avaliação revisada por pares.

A lacuna, portanto, não está na ausência de discussão sobre os *trade-offs* dessa camada, mas na ausência de **medição controlada** deles: não foram encontrados estudos que

comparem, sob as mesmas condições, uma aplicação com e sem a camada de integração, quantificando ao mesmo tempo o benefício mais alegado da camada — a facilidade de trocar de provedor — e o preço que ela cobra em energia. Sem essa medição, a decisão de adotar ou não uma camada de *harness* fica apoiada em argumentos de princípio ou em alegações de fornecedores.

Diante disso, este trabalho busca responder à seguinte questão de pesquisa: *a introdução de uma camada arquitetural intermediária entre uma aplicação web e um provedor de IA reduz o custo de substituição do provedor, e qual é o custo energético dessa redução?*

Parte-se da hipótese de que a camada reduz o custo de substituição e cobra por isso um custo energético; essa hipótese é tratada como algo a testar, e não como pressuposto. Caso a camada reduza o custo de substituição sem acréscimo relevante de energia, ou caso acrescente custo energético sem reduzir o esforço de substituição, ambos os resultados são informativos para quem precisa tomar a decisão arquitetural.

1.3 Justificativa

Do ponto de vista prático, desenvolvedores e arquitetos que integram LLMs a aplicações *web* decidem, explícita ou implicitamente, entre acoplar o código ao SDK de um provedor ou investir em uma camada de abstração, própria ou oferecida por um *framework*. Em um mercado no qual a liderança entre provedores mudou em menos de dois anos (MENLO VENTURES, 2025), o custo de trocar de provedor deixa de ser hipotético; e, em um cenário de demanda crescente por energia nos centros de dados (INTERNATIONAL ENERGY AGENCY, 2025), qualquer sobrecarga paga a cada requisição é multiplicada pelo volume de uso. Evidência quantitativa sobre os dois lados dessa decisão permite tomá-la com base em dados, e não apenas em princípios de projeto ou em material promocional.

Do ponto de vista científico, o trabalho contribui para a arquitetura de *software* de sistemas com IA ao submeter a avaliação experimental uma decisão que, até aqui, foi discutida principalmente de forma qualitativa (LIU *et al.*, 2025), e aproxima a Engenharia de Software da agenda de *Green AI* (SCHWARTZ *et al.*, 2020) ao tratar o consumo de energia como atributo de qualidade de uma decisão arquitetural, e não apenas do modelo. O estudo segue os princípios de experimentação controlada em Engenharia de Software sistematizados por Wohlin *et al.* (2024), comparando duas versões funcionalmente equivalentes de uma mesma aplicação: uma integrada ao provedor por meio da camada e outra integrada diretamente.

O recorte em duas dimensões é deliberado. A substituíbilidade do provedor representa o benefício mais associado à camada, e o custo energético, o preço que ela cobra; juntos, formam um compromisso fechado sobre a mesma decisão. Outros atributos usualmente atribuídos a uma camada desse tipo, como a recuperação de falhas e a auditabilidade, serão implementados por serem pré-requisitos técnicos do experimento, mas não serão avaliados. Por isso, a conclusão do trabalho se limita a responder se a camada compensa pelo desacoplamento que proporciona, e não se ela compensa de modo geral.

Por fim, o trabalho é viável no prazo de um trabalho de conclusão de curso. A medição energética apoia-se em protocolo de telemetria de *hardware* já empregado pelo autor em trabalho anterior (BARROS *et al.*, 2026), e o experimento utiliza infraestrutura com GPU disponível na instituição e modelos abertos executados localmente.

1.4 Objetivos

1.4.1 *Objetivo geral*

Avaliar experimentalmente o *trade-off* entre desacoplamento e eficiência energética introduzido por uma camada arquitetural intermediária (*harness*) entre aplicações *web* e provedores de modelos de IA, em comparação com a integração direta ao provedor.

1.4.2 *Objetivos específicos*

- a) projetar uma camada de *harness* para a integração de modelos de IA em aplicações *web*, delimitando suas responsabilidades com base em padrões arquiteturais publicados;
- b) quantificar o custo de substituição do provedor de IA em uma aplicação *web* com e sem a camada de *harness*;
- c) mensurar o custo energético que a camada de *harness* acrescenta à execução de requisições simples e agênticas a um modelo de IA;
- d) identificar em que condições a redução do custo de substituição compensa o custo energético da camada.

REFERÊNCIAS

- BARROS, L. A. M.; SILVA, V. M. A. da; SANTOS, T. A. dos; VIANA, G. M. R.; GOMES DE B. FILHO, B. Single prompt vs. debate multiagente: uma análise de telemetria de hardware em tarefas de contexto longo com Llama 3.2 2B. In: **Anais do LIII Seminário Integrado de Software e Hardware (SEMISH)**. Porto Alegre: Sociedade Brasileira de Computação, 2026.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. **Software Architecture in Practice**. 4. ed. Boston: Addison-Wesley, 2021.
- FERNANDEZ, J.; NA, C.; TIWARI, V.; BISK, Y.; LUCCIONI, S.; STRUBELL, E. Energy considerations of large language model inference and efficiency optimizations. In: **Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)**. Vienna: Association for Computational Linguistics, 2025. p. 32556–32569.
- IFATH, M. M. A.; HAQUE, I. Characterizing performance–energy trade-offs of large language models in multi-request workflows. In: **Abstracts of the 2026 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems**. New York: ACM, 2026.
- INTERNATIONAL ENERGY AGENCY. **Energy and AI**. Paris, 2025. Disponível em: <<https://www.iea.org/reports/energy-and-ai>>. Acesso em: 9 out. 2026.
- ISSARNY, V.; CAPORUSCIO, M.; GEORGANTAS, N. A perspective on the future of middleware-based software engineering. In: **Future of Software Engineering (FOSE '07)**. Minneapolis: IEEE, 2007. p. 244–258.
- KANDOGAN, E.; BHUTANI, N.; ZHANG, D.; CHEN, R. L.; GURAJADA, S.; HRUSCHKA, E. Orchestrating agents and data for enterprise: A blueprint architecture for compound AI. In: **2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW)**. Piscataway: IEEE, 2025.
- KAUR, K.; SHARMA, S.; KAHLON, K. S. Interoperability and portability approaches in inter-connected clouds: A review. **ACM Computing Surveys**, v. 50, n. 4, p. 1–40, 2017.
- LIU, Y.; LO, S. K.; LU, Q.; ZHU, L.; ZHAO, D.; XU, X.; HARRER, S.; WHITTLE, J. Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents. **Journal of Systems and Software**, v. 220, p. 112278, 2025.
- LUCCIONI, S.; JERNITE, Y.; STRUBELL, E. Power hungry processing: Watts driving the cost of AI deployment? In: **Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency**. New York: ACM, 2024. p. 85–99.
- MENLO VENTURES. **2025 Mid-Year LLM Market Update**. 2025. Disponível em: <<https://menlovc.com/perspective/2025-mid-year-llm-market-update/>>. Acesso em: 9 out. 2026.
- MO, D.; CORDINGLY, R.; CHINN, D.; LLOYD, W. Addressing serverless computing vendor lock-in through cloud service abstraction. In: **2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)**. Piscataway: IEEE, 2023.
- PARNAS, D. L. On the criteria to be used in decomposing systems into modules. **Communications of the ACM**, v. 15, n. 12, p. 1053–1058, 1972.

SAPKOTA, R.; ROUMELIOTIS, K. I.; KARKEE, M. AI agents vs. agentic AI: A conceptual taxonomy, applications and challenges. **Information Fusion**, v. 126, p. 103599, 2026.

SCHWARTZ, R.; DODGE, J.; SMITH, N. A.; ETZIONI, O. Green AI. **Communications of the ACM**, v. 63, n. 12, p. 54–63, 2020.

SCULLEY, D.; HOLT, G.; GOLOVIN, D.; DAVYDOV, E.; PHILLIPS, T.; EBNER, D.; CHAUDHARY, V.; YOUNG, M.; CRESPO, J.-F.; DENNISON, D. Hidden technical debt in machine learning systems. In: **Advances in Neural Information Processing Systems 28 (NIPS 2015)**. Red Hook: Curran Associates, 2015. p. 2503–2511.

STACK OVERFLOW. **2025 Developer Survey: AI**. 2025. Disponível em: <<https://survey.stackoverflow.co/2025/ai>>. Acesso em: 9 out. 2026.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in NLP. In: **Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics**. Florence: Association for Computational Linguistics, 2019. p. 3645–3650.

VELLAISAMY, P.; TRIPATHI, S.; NATARAJAN, V.; THENARASU, S. S.; BLANTON, S.; SHEN, J. P. TaxBreak: Unmasking the hidden costs of LLM inference through overhead decomposition. In: **2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)**. Piscataway: IEEE, 2026.

WOHLIN, C.; RUNESON, P.; HÖST, M.; OHLSSON, M. C.; REGNELL, B.; WESSLÉN, A. **Experimentation in Software Engineering**. 2. ed. Berlin, Heidelberg: Springer, 2024.

ZHU, Y.; LIU, L.; YU, J.; ZHANG, D. LLM-based multi-agent orchestration: A survey of frameworks, communication protocols, and emerging patterns. **Future Internet**, v. 18, n. 6, p. 326, 2026.